

Module	Task	Assessment type
Artificial Intelligence and Machine Learning	1	Individual Report

Tutor: Mr. Shiv Kumar Yadav

Name: Dipendra Roka

Group: L6CG18

Canvas ID:2418541

Submission Date: 2026/5/10

Github link:

Acknowledgement

I would like to sincerely thank everyone who supported me in completing this report. My deepest appreciation goes to Mr. Siman Giri, the module leader, and Mr. Shiv Kumar Yadav, my tutor, for their continuous guidance, constructive feedback, and encouragement throughout this process. Their assistance has played a significant role in my learning and academic growth. I am also grateful to the University and the College for offering this module, which has greatly contributed to enhancing my knowledge and skills.

Table of Contents

<i>Part I – Long Question</i>	4
Unsupervised Learning in Digital Payment Platforms	4
<i>Part II – Short Questions</i>	5
1. Overfitting and underfitting.....	5
2. Neural Network Architecture	6
<i>References</i>	7

Part I – Long Question

Unsupervised Learning in Digital Payment Platforms

As a Data Scientist working for a digital payment platform such as eSewa, unsupervised learning can help solve several business problems even when labeled data is limited. One important application is customer segmentation. Digital payment platforms have users with different spending habits, transaction frequencies, and service preferences. Understanding these behaviors helps companies create targeted marketing campaigns and personalized offer (Gonçalves, 2024).

To solve this problem, clustering techniques such as K-Means can be used. The algorithm groups users with similar transaction patterns together (Geeksforgeeks, 2025). For example, one cluster may contain students making small daily payments, while another may represent business users handling large transfers. These insights can be integrated into recommendation systems or marketing dashboards in batch processing systems. A challenge of K-Means is selecting the correct number of clusters, since poor selection may produce inaccurate customer groups.

Another valuable application is fraud detection. Payment systems process thousands of transactions daily, making manual monitoring difficult. Since fraudulent transactions are rare and often unlabeled, anomaly detection methods are useful. Autoencoders can learn normal transaction behavior and identify unusual activities that differ from typical patterns. For instance, a sudden large transaction from a new location may be flagged as suspicious.

The model outputs can be connected to real-time fraud monitoring systems that automatically trigger alerts or temporarily pause suspicious transactions for verification. This improves transaction security and reduces financial risk. However, anomaly detection systems may sometimes generate false positives, where legitimate users are incorrectly flagged. This can affect user experience if not handled carefully.

Overall, unsupervised learning provides practical solutions for extracting useful insights from large-scale transaction data without relying heavily on labeled datasets. These methods support business growth, improve customer experience, and strengthen platform security.

Part II – Short Questions

1. Overfitting and underfitting

Overfitting happens when a machine learning model learns the training data too closely, including noise and unnecessary details. As a result, it performs very well on training data but poorly on new unseen data. Underfitting occurs when the model is too simple to learn important patterns from the dataset. In this case, the model performs poorly on both training and testing data.

Both problems reduce generalization performance, which means the model cannot make accurate predictions on real-world data. A balanced model should learn useful patterns without memorizing the dataset.

In machine learning, overfitting often occurs in very complex models with too many parameters, while underfitting usually happens when the model is too simple for the problem. Proper model selection, regularization, and sufficient training data help improve generalization and reduce these issues.

2. Neural Network Architecture

CNN vs RNN

Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) are designed for different types of data. CNNs mainly work with image-related data where spatial information is important. They use convolution layers to automatically detect features such as edges, shapes, and textures. In CNNs, the same filters are shared across the image, reducing the number of parameters.

RNNs are designed for sequential data such as text, speech, or time-series information. Unlike CNNs, RNNs process data step by step while maintaining information from previous inputs through hidden states (Xu, 2024). This allows the network to learn sequence patterns and context.

CNNs are commonly used in image classification tasks such as facial recognition or medical image analysis (Mukherjee & Olu-Ipinlaye, 2025). RNNs are preferred for language translation, chatbots, and stock price prediction because they can handle sequential dependencies (Pillai, 2019).

Training deep learning models also comes with challenges. One issue is vanishing or exploding gradients, where gradients become too small or too large during backpropagation. This makes training unstable. Techniques such as gradient clipping and batch normalization help reduce this problem (Blueprint, 2025).

Another challenge is overfitting, where the model memorizes training data instead of learning general patterns. Methods like early stopping and dropout help improve generalization by preventing excessive learning from training data alone.

References

- Xu, Z. (2024, March 18). *Baeldung*. From Baeldung: <https://www.baeldung.com/cs/lstm-vanishing-gradient-prevention>
- Mukherjee, S., & Olu-Ipinlaye. (2025, April 29). *Digitalocean*. From <https://www.digitalocean.com/community/tutorials/filters-in-convolutional-neural-networks>
- Pillai, P. (2019, July 17). *Medium*. From <https://medium.com/@pranavp802/recurrent-neural-networks-the-vanishing-gradient-problem-and-lstms-3ac0ad8aff10>
- Blueprint, T. (2025, February 25). *Medium*. From <https://medium.com/architects-of-intelligence/deep-learning-essentials-a-beginners-guide-to-batch-normalization-dropout-and-optimizers-04210420ff3c>